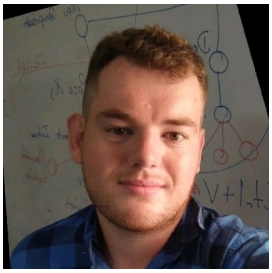


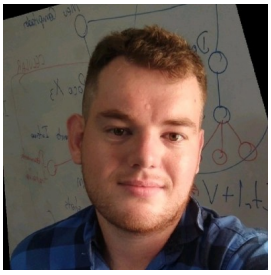
Universidade de Rio Verde

Sergio Souza Novak



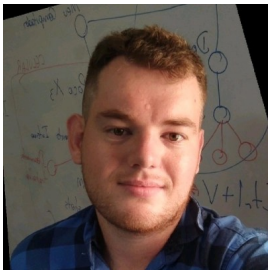
Sergio Souza Novak é Mestre em Ciências Agrárias pelo Instituto Federal Goiano. Atua nas áreas de Banco de Dados, Inteligência Artificial e Desenvolvimento de Software, com experiência acadêmica e industrial e publicações científicas aplicadas à agricultura digital.

Ronaldo



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit.

Leandro



Leandro possui experiência em computação aplicada, análise de sistemas e desenvolvimento tecnológico, participando de projetos acadêmicos e educacionais.

Sumário

Capítulo 1

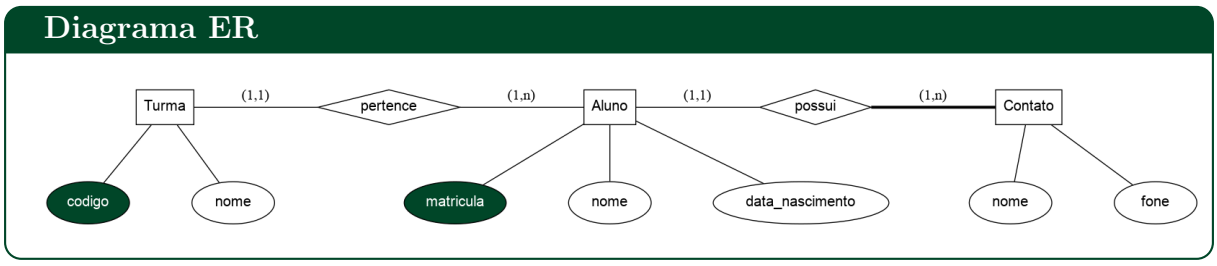
Modelo Entidade Relacionamento

O desenvolvimento de um Banco de Dados para uma aplicação, antes da implementação técnica, exige uma compreensão clara e estruturada dos dados que serão manipulados. Nesse contexto, o **Diagrama Entidade-Relacionamento (DER)** desempenha um papel fundamental no processo de modelagem de dados. O Diagrama Entidade-Relacionamento é um **modelo conceitual**, utilizado para representar, de forma abstrata e organizada, a estrutura das informações de um sistema. Ele está mais próximo da visão que o usuário possui dos dados, pois descreve os elementos do mundo real que serão armazenados e como eles se conectam entre si.

Por meio do DER, é possível identificar e definir:

- Quais dados devem ser armazenados pela aplicação;
- As entidades que representam objetos ou conceitos do domínio do problema;
- Os atributos que caracterizam essas entidades;
- Como esses dados se relacionam entre si.

Dessa forma, o modelo Entidade-Relacionamento serve como base para a construção do modelo lógico e, posteriormente, do banco de dados físico. Ele contribui para reduzir ambiguidades, melhorar a comunicação entre desenvolvedores e usuários e garantir que o sistema atenda corretamente aos requisitos estabelecidos. Ao longo deste livro, serão apresentados os conceitos fundamentais do modelo Entidade-Relacionamento, suas notações, regras de construção e exemplos práticos, permitindo ao leitor compreender e aplicar essa importante ferramenta no projeto de bancos de dados. Abaixo está ilustrado um diagrama desse tipo:



Antes de entendermos o diagrama, vejamos a Tabela **??**. A Tabela **??** apresenta os principais elementos que compõem o Diagrama Entidade-Relacionamento, destacando suas representações gráficas e respectivas funções no modelo conceitual. Os retângulos

identificam os conjuntos de entidades, as elipses representam os atributos que descrevem suas características, os losangos indicam os relacionamentos estabelecidos entre entidades e as linhas evidenciam as conexões estruturais entre esses componentes. Em conjunto, esses elementos permitem a representação organizada e formal das informações de um domínio, servindo como base para a modelagem lógica e posterior implementação em banco de dados.

Elemento Gráfico	**Descrição**
Retângulo	Representa um *conjunto de entidades*. Uma entidade corresponde a um objeto ou conceito do mundo real sobre o qual se deseja armazenar informações. Exemplos incluem *Aluno*, *Turma* ou *Produto*. Cada instância da entidade possui existência própria dentro do sistema.
Elipse	Representa um *atributo*. Os atributos descrevem características ou propriedades das entidades ou dos relacionamentos. Exemplos incluem *nome*, *matrícula*, *data_nascimento*. Um atributo pode ser simples, composto, multivalorado ou chave.
Losango	Representa um *relacionamento*. O relacionamento descreve a associação existente entre dois ou mais conjuntos de entidades. Por exemplo, o relacionamento *pertence* pode associar a entidade *Aluno* à entidade *Turma*.
Linha	Representa a ligação entre elementos do diagrama. As linhas conectam atributos às entidades e entidades aos relacionamentos, indicando participação e associação estrutural no modelo conceitual.

Tabela 1.1: Elementos fundamentais do Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento apresentado modela um cenário acadêmico composto pelas entidades *Turma*, *Aluno* e *Contato*. O modelo entidade-relacionamento tem por base a percepção de que o mundo real é formado por conjunto de objetos chamados entidades e pelo conjunto dos relacionamentos entre esses objetos. Retângulos, que representam os conjuntos de entidades; Elipses, que representam os atributos; Losangos, que representam os relacionamentos entre os conjuntos de entidades; Linhas, que unem os atributos aos conjuntos de entidades e o conjunto de entidades aos seus relacionamentos.

O modelo tem como objetivo representar a organização de alunos em turmas e o cadastro de contatos associados a cada aluno, evidenciando as estruturas de dados e as restrições de cardinalidade existentes no domínio analisado. A entidade **Turma** representa as unidades organizacionais às quais os alunos estão vinculados. Ela possui dois atributos: *codigo*, que atua como chave primária e identifica unicamente cada turma no sistema, e *nome*, que descreve a denominação da turma. A presença de uma chave primária garante a integridade da identificação das instâncias dessa entidade. A entidade **Aluno** representa os estudantes vinculados às turmas. Seus atributos são *matricula*,

nome e *data_nascimento*. O atributo *matricula* é definido como chave primária, sendo responsável por identificar de forma única cada aluno. Os demais atributos armazenam informações descritivas e complementares necessárias para caracterizar cada estudante.

A entidade **Contato** representa os dados de contato associados aos alunos. Ela possui os atributos *nome* e *fone*, que registram, respectivamente, o nome do contato e o número de telefone correspondente. Essa entidade permite armazenar múltiplos registros vinculados a um mesmo aluno. O relacionamento **pertence** estabelece a associação entre as entidades *Aluno* e *Turma*. A cardinalidade indicada no diagrama demonstra que cada aluno deve estar obrigatoriamente vinculado a exatamente uma turma, enquanto uma turma pode estar associada a um ou vários alunos. Trata-se, portanto, de um relacionamento do tipo um-para-muitos (1:N), com participação obrigatória do lado da entidade Aluno. O relacionamento **possui** associa a entidade *Aluno* à entidade *Contato*. A cardinalidade apresentada indica que cada aluno possui pelo menos um contato, podendo possuir vários, enquanto cada contato está vinculado a exatamente um aluno. Esse relacionamento também é classificado como um relacionamento do tipo um-para-muitos (1:N), com participação obrigatória da entidade Contato em relação ao aluno ao qual está associado.

De maneira geral, o modelo representa de forma consistente as restrições do domínio acadêmico considerado, garantindo identificação única das entidades principais e estabelecendo vínculos obrigatórios entre alunos e turmas, bem como entre alunos e seus respectivos contatos. A estrutura apresentada é adequada para posterior transformação em modelo relacional, preservando as chaves primárias e as dependências representadas pelas cardinalidades.

1.1 Conceito de Atributos

No modelo Entidade-Relacionamento, os **atributos** são as propriedades que descrevem as características de uma entidade ou de um relacionamento. Eles representam as informações que desejamos armazenar sobre cada ocorrência de uma entidade no sistema.

Cada entidade possui um conjunto próprio de atributos que definem seus dados. Por exemplo, a entidade **Professor** pode possuir atributos como matrícula, nome e telefone. A entidade **Disciplina** pode possuir atributos como código e nome. Já a entidade **Aluno** pode possuir atributos como matrícula, nome e data de nascimento.

Os atributos podem ser classificados em diferentes tipos conceituais:

- **Atributo simples**: não pode ser subdividido em partes menores (exemplo: matrícula).
- **Atributo composto**: pode ser dividido em subpartes (exemplo: um endereço poderia ser dividido em rua, número e cidade).
- **Atributo multivalorado**: pode assumir mais de um valor para a mesma entidade (exemplo: múltiplos telefones).
- **Atributo identificador (chave)**: atributo responsável por identificar unicamente cada ocorrência de uma entidade.

Os atributos são fundamentais para dar significado às entidades, pois sem eles não seria possível armazenar ou diferenciar as instâncias no banco de dados.

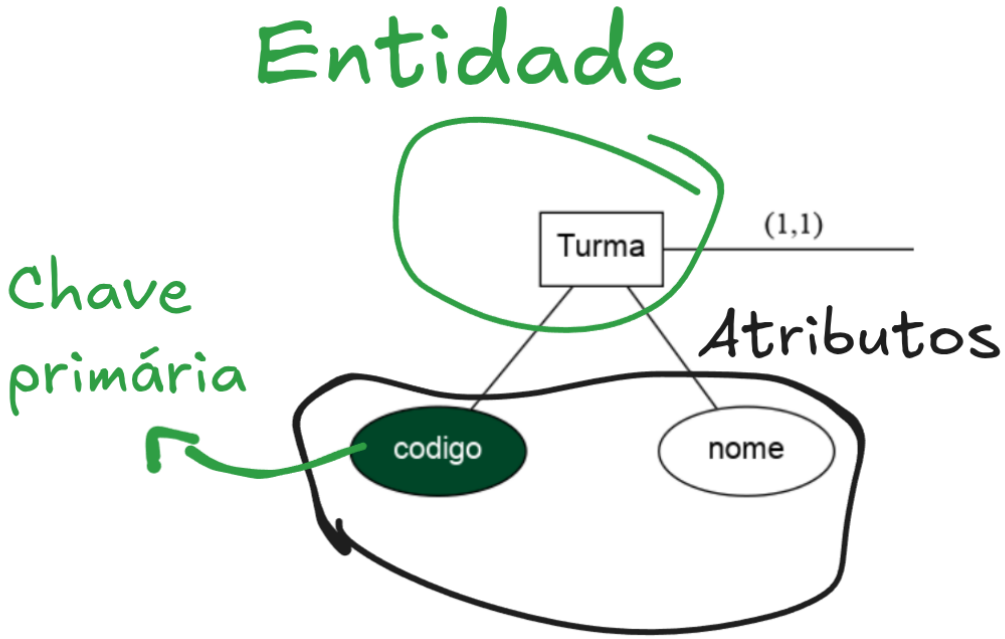


Figura 1.1: Atributos do Diagrama ER.

1.2 Chave Primária

Em um modelo de banco de dados relacional, a **chave primária** é o atributo, ou conjunto de atributos, responsável por identificar de maneira única cada registro de uma tabela. Isso significa que não podem existir duas linhas com o mesmo valor para a chave primária, garantindo que cada tupla seja distinta dentro da relação.

A principal função da chave primária é assegurar a integridade dos dados, evitando duplicidades e permitindo que cada registro seja referenciado de forma precisa. Para cumprir esse papel, ela deve apresentar algumas características fundamentais: seus valores devem ser únicos em toda a tabela, não podem assumir o valor `NULL`, devem ser preferencialmente estáveis ao longo do tempo (ou seja, não devem sofrer alterações frequentes) e devem conter apenas os atributos estritamente necessários para identificar o registro, preservando o princípio da minimalidade.

Como exemplo, considere a entidade *Cliente*, composta pelos atributos id_cliente, nome e telefone. Nesse caso, o atributo `id_cliente` é definido como chave primária, pois identifica exclusivamente cada cliente no sistema, independentemente de possíveis repetições de nome ou telefone.

Em determinadas situações, a chave primária pode ser formada por mais de um atributo, caracterizando uma **chave primária composta**. Isso ocorre quando nenhum atributo isoladamente é capaz de garantir a identificação única do registro. Um exemplo comum é uma tabela que representa um relacionamento muitos-para-muitos entre as entidades *Aluno* e *Disciplina*. Nessa situação, a combinação dos atributos `id_aluno` e `id_disciplina` forma a chave primária, pois apenas o conjunto desses dois valores assegura a unicidade de cada registro.

Assim, a definição adequada da chave primária é essencial no modelo relacional, pois possibilita o estabelecimento correto de relacionamentos entre tabelas, previne inconsis-

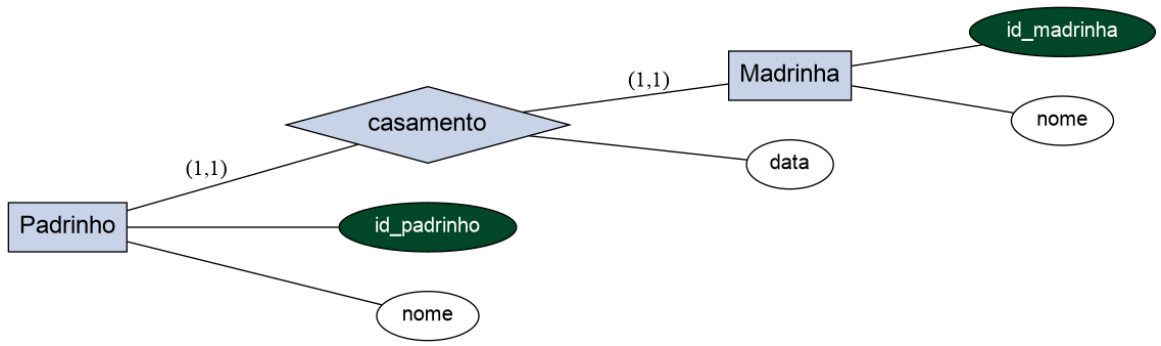
tências e mantém a integridade referencial do banco de dados. Por esse motivo, toda tabela deve possuir uma chave primária claramente definida.

1.3 Conceito de Relacionamentos

Os **relacionamentos** representam as associações existentes entre duas ou mais entidades dentro do modelo. Eles indicam como as entidades se conectam e interagem no domínio do problema. Um relacionamento pode ser classificado de acordo com sua **cardinalidade**, que define o número mínimo e máximo de ocorrências de uma entidade que podem estar associadas a outra. As cardinalidades mais comuns são:

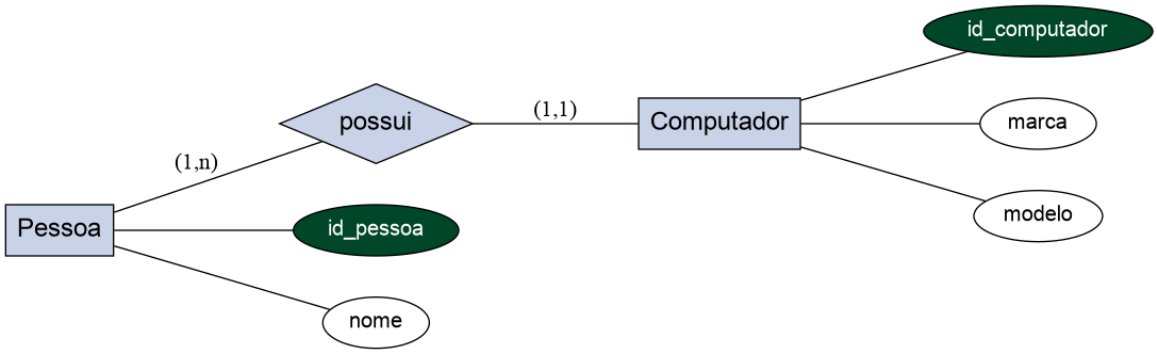
- **Um-para-um (1:1)**: uma ocorrência de uma entidade está associada a apenas uma ocorrência da outra.
- **Um-para-muitos (1:N)**: uma ocorrência de uma entidade pode estar associada a várias ocorrências da outra.
- **Muitos-para-muitos (N:M)**: várias ocorrências de uma entidade podem estar associadas a várias ocorrências da outra.

Diagrama ER



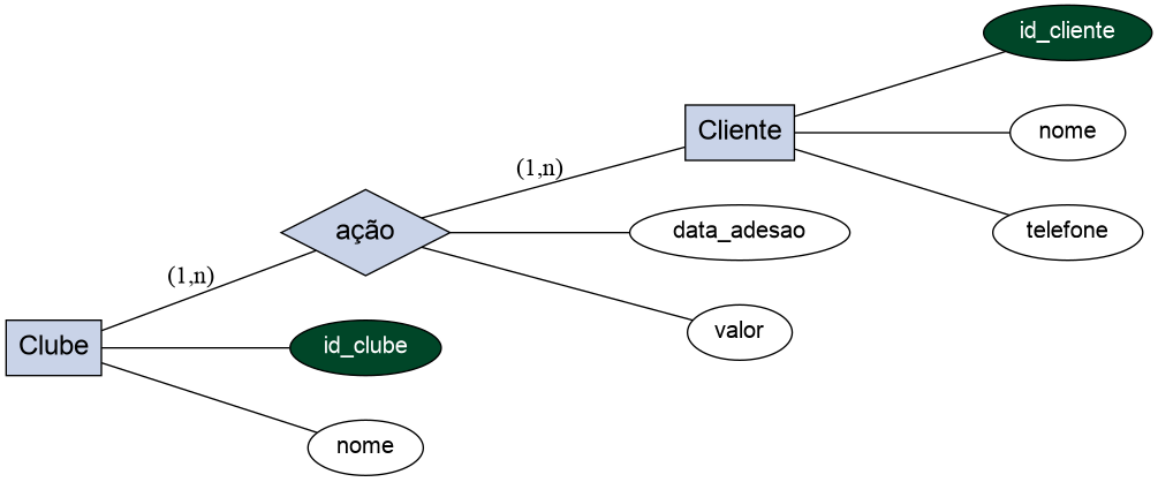
No primeiro diagrama, é apresentado um relacionamento do tipo um-para-um (1:1) entre as entidades Padrinho e Madrinha, mediado pelo relacionamento casamento. A cardinalidade (1,1) em ambos os lados indica que cada padrinho está associado a exatamente uma madrinha, e cada madrinha está associada a exatamente um padrinho dentro do contexto modelado. Isso caracteriza uma associação exclusiva entre as duas entidades. Cada entidade possui um atributo identificador destacado como chave primária id_padrinho e id_madrinha, cuja função é garantir a identificação única de cada ocorrência. Além disso, ambas possuem atributos descritivos, como nome. O relacionamento casamento também possui um atributo próprio, data, que armazena uma informação específica da associação entre padrinho e madrinha, demonstrando que relacionamentos também podem possuir atributos quando é necessário registrar características do vínculo entre as entidades.

Diagrama ER



No segundo diagrama, é representado um relacionamento do tipo um-para-muitos (1:N) entre as entidades Pessoa e Computador, por meio do relacionamento possui. A cardinalidade (1,n) do lado de Pessoa indica que uma pessoa pode possuir vários computadores, enquanto a cardinalidade (1,1) do lado de Computador indica que cada computador pertence a exatamente uma pessoa. Esse tipo de modelagem é comum quando um elemento principal pode estar associado a várias ocorrências de outro elemento dependente. A entidade Pessoa possui como chave primária o atributo id_pessoa, além do atributo descritivo nome. A entidade Computador possui id_computador como chave primária, além dos atributos marca e modelo. Neste caso, o relacionamento não possui atributos próprios, pois a associação "possui" não necessita armazenar informações adicionais além da ligação entre as entidades.

Diagrama ER



No terceiro diagrama, é apresentado um relacionamento do tipo muitos-para-muitos (N:M) entre as entidades Clube e Cliente, por meio do relacionamento ação. A cardinalidade $1, n$ em ambos os lados indica que um clube pode estar associado a vários clientes e que um cliente pode participar de vários clubes simultaneamente. Esse tipo de relacionamento representa uma associação múltipla em ambas as direções, sendo bastante comum em sistemas reais.
A entidade Clube possui id_clube como chave primária e nome como atributo descritivo, enquanto a entidade Cliente possui id_cliente como chave primária, além de nome e telefone. Diferentemente do caso anterior, o relacionamento ação possui atributos próprios, como data_adesao e valor, que armazenam informações específicas da participação do cliente no clube.
Isso demonstra que, em relacionamentos muitos-para-muitos, é frequente a necessidade de registrar dados adicionais referentes à própria associação.

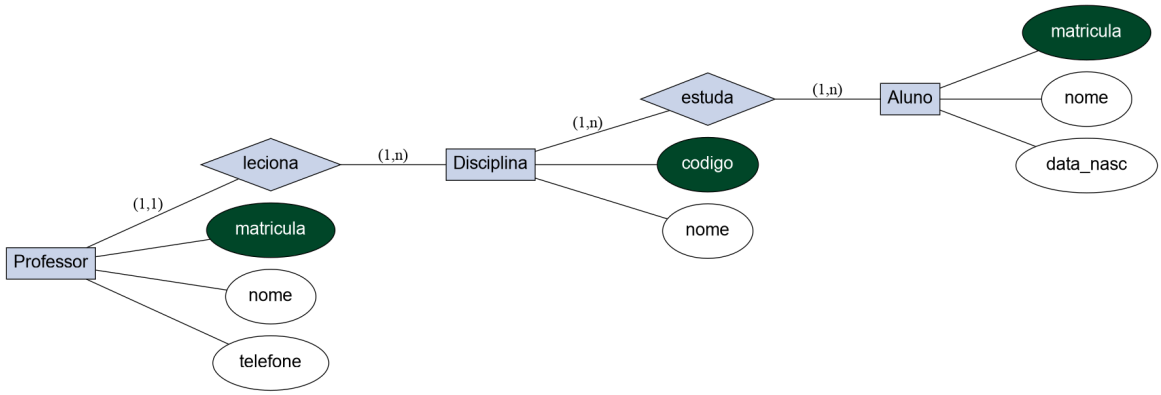
Além disso, os relacionamentos podem possuir atributos próprios, quando é necessário armazenar informações específicas sobre a associação entre as entidades. Portanto, enquanto as entidades representam os objetos principais do domínio e os atributos descrevem suas características, os relacionamentos estabelecem as ligações estruturais que organizam o modelo de dados.

1.4 Exemplo de diagrama ER

O diagrama Entidade-Relacionamento apresentado é composto por três entidades principais: **Professor**, **Disciplina** e **Aluno**. Essas entidades representam os objetos fundamentais do domínio acadêmico modelado. A entidade **Professor** representa os docentes responsáveis pela oferta das disciplinas. Já a entidade **Disciplina** representa as matérias ofertadas na instituição. Por sua vez, a entidade **Aluno** representa os estudantes matriculados na instituição. Entre essas entidades existem dois relacionamentos principais. O primeiro relacionamento, denominado **leciona**, conecta as entidades **Professor** e **Disciplina**. A cardinalidade indica que cada professor participa do relacionamento com cardinalidade (1,1), enquanto cada disciplina participa com cardinalidade (1,n). Isso significa que um professor está associado a exatamente uma ocorrência no relacionamento, enquanto uma disciplina pode estar associada a uma ou várias ocorrências conforme definido no modelo. O segundo relacionamento, denominado **estuda**, conecta as entidades **Disciplina** e **Aluno**. A cardinalidade (1,n) em ambos os lados indica que uma disciplina pode estar relacionada a vários alunos e que um aluno pode estar relacionado a várias disciplinas, caracterizando um relacionamento do tipo muitos-para-muitos.

Assim, o modelo descreve a estrutura básica de um sistema acadêmico no qual professores lecionam disciplinas e alunos estudam disciplinas, estabelecendo as ligações essenciais entre os principais elementos do domínio.

Diagrama ER



1.5 Atividade prática de MER

1.5.1 Objetivo

O objetivo desta atividade é desenvolver a capacidade de análise de requisitos e modelagem conceitual de um banco de dados por meio da construção de um **Diagrama Entidade-Relacionamento (DER)** para um sistema realista de controle de ordenha em uma fazenda leiteira.

1.5.2 Contexto do Sistema



Figura 1.2: Uma fazenda da cidade de Rio Verde. Fonte: Instagram @fazendanycolle

Uma fazenda leiteira deseja informatizar o controle de suas atividades de ordenha. Atualmente, os registros são feitos manualmente, o que dificulta o acompanhamento da produção, da saúde dos animais e do desempenho dos funcionários. O sistema deverá armazenar informações sobre as vacas, as ordenhas realizadas, os funcionários responsáveis, os equipamentos utilizados e a produção de leite obtida em cada ordenha.

1.5.3 Descrição dos Requisitos

Abaixo são detalhados os requisitos levantados para a modelagem do sistema:

Vacas

Cada vaca da fazenda deve ser identificada por um código único. Além disso, devem ser armazenadas informações como:

- Nome ou identificação visual;
- Data de nascimento;
- Raça, Peso e Status de saúde.

Funcionários e Supervisão

A fazenda possui funcionários com diferentes atribuições:

- **Funcionários de Campo:** Registrados por código, nome completo, função (ordenhador, etc.) e telefone.
- **Supervisor:** Responsável por acompanhar o trabalho. Registra-se código, nome, telefone e turno de supervisão.
- **Veterinário:** Responsável pela saúde clínica. Registra-se código, nome, registro profissional (CRMV) e telefone.

Processo de Ordenha

A ordenha ocorre diariamente e pode acontecer mais de uma vez por dia para a mesma vaca. Cada registro de ordenha deve conter: **Data, Horário e Quantidade de leite (litros)**.

1.5.4 Regras de Negócio e Relacionamentos

Para facilitar o entendimento das cardinalidades, o produtor descreveu a rotina da seguinte forma:

Rotina de Ordenha

Cada vaca da fazenda é ordenhada ao longo do tempo. Em alguns períodos, a mesma vaca pode ser ordenhada mais de uma vez no mesmo dia. Sempre que ocorre uma ordenha, ela diz respeito a uma única vaca.

Trabalho dos Funcionários

Os funcionários são responsáveis por realizar as ordenhas. Um mesmo funcionário pode ordenhar várias vacas ao longo do dia. No entanto, toda ordenha precisa ter alguém responsável por realizá-la.

Uso dos Equipamentos

Para realizar a ordenha, podem ser usados um ou vários equipamentos (manuais, automáticos ou robóticos). Os equipamentos podem ser utilizados em diferentes ordenhas ou ficar parados para manutenção.

Cuidado com a Saúde

O veterinário pode atender várias vacas ao longo do tempo e, dependendo do caso, a mesma vaca pode precisar de mais de um atendimento em momentos diferentes.

Visão do Proprietário

O proprietário não participa da ordenha diretamente, mas acompanha os relatórios de produção, desempenho da equipe e utilização de equipamentos para tomada de decisão.

1.5.5 Atividades Propostas

Com base na descrição acima, realize a modelagem conceitual seguindo os passos:

1. Identifique todas as **entidades** do sistema.
2. Para cada entidade, defina seus **atributos** e a **chave primária**.
3. Modele os **relacionamentos** (ex: Vaca *sofre* Ordenha).
4. Defina as **cardinalidades** (1:1, 1:N, N:M) e a **participação** (total ou parcial).
5. Identifique se há necessidade de atributos nos relacionamentos.

Instruções de Entrega: O Diagrama Entidade-Relacionamento deve ser feito de forma legível (à mão ou ferramenta digital como brModelo/Lucidchart). O arquivo final deve ser convertido para **PDF** e conter apenas o diagrama finalizado.

1.5.6 Critérios de Avaliação

A avaliação considerará a correta identificação das entidades, adequação das chaves primárias, precisão das cardinalidades frente às regras de negócio e a organização visual do diagrama.

Capítulo 2

Introdução ao SQL e ao pgAdmin 4

SQL é a sigla para *Structured Query Language* (Linguagem de Consulta Estruturada). Trata-se de uma linguagem padrão utilizada para manipular e gerenciar bancos de dados relacionais. Um **banco de dados relacional** organiza as informações em **tabelas**, compostas por **linhas** (registros) e **colunas** (atributos). Esse modelo foi formalizado por Edgar F. Codd na década de 1970 e tornou-se o paradigma dominante em sistemas de gerenciamento de dados. A linguagem SQL permite realizar diversas operações fundamentais no gerenciamento de bancos de dados relacionais, tais como a criação de bancos de dados e tabelas, a inserção de novos registros, a consulta de informações específicas por meio de critérios definidos, a atualização de dados já existentes, a remoção de registros e o controle de permissões de acesso dos usuários ao sistema.É importante destacar que o SQL é utilizado por diversos Sistemas Gerenciadores de Banco de Dados (SGBDs), como o *PostgreSQL*, o MySQL, o Oracle Database e o SQL Server.

De forma didática, podemos dividir os comandos SQL em categorias:

- **DDL (Data Definition Language)**: comandos de definição de estrutura, como `CREATE`, `ALTER` e `DROP`.
- **DML (Data Manipulation Language)**: comandos de manipulação de dados, como `INSERT`, `UPDATE`, `DELETE`.
- **DQL (Data Query Language)**: comandos de consulta, como `SELECT`.
- **DCL (Data Control Language)**: comandos de controle de acesso, como `GRANT` e `REVOKE`.

Um exemplo simples de consulta em SQL é:

SQL

```sql
SELECT nome, idade
FROM alunos
WHERE idade > 18;
```

Nesse exemplo, o comando solicita a exibição dos campos `nome` e `idade` da tabela `alunos`, considerando apenas os registros em que a idade seja maior que 18 anos. Esse exemplo poderia retornar a tabela abaixo, dependendo dos dados armazenados:

Nome	Idade	CPF
Ana Silva	20	123.456.789-00
Carlos Lima	22	987.654.321-00
Mariana Souza	19	456.789.123-00

2.1 pgAdmin 4

O pgAdmin 4 é uma ferramenta gráfica utilizada para administrar bancos de dados do sistema PostgreSQL. Ele funciona como uma interface visual que facilita a interação com o banco de dados, especialmente para usuários iniciantes, pois permite executar tarefas administrativas sem a necessidade de memorizar todos os comandos da linguagem SQL. A Figura **??** apresenta a interface do pgAdmin 4, destacando os principais painéis e recursos disponíveis para gerenciamento de bancos de dados PostgreSQL.

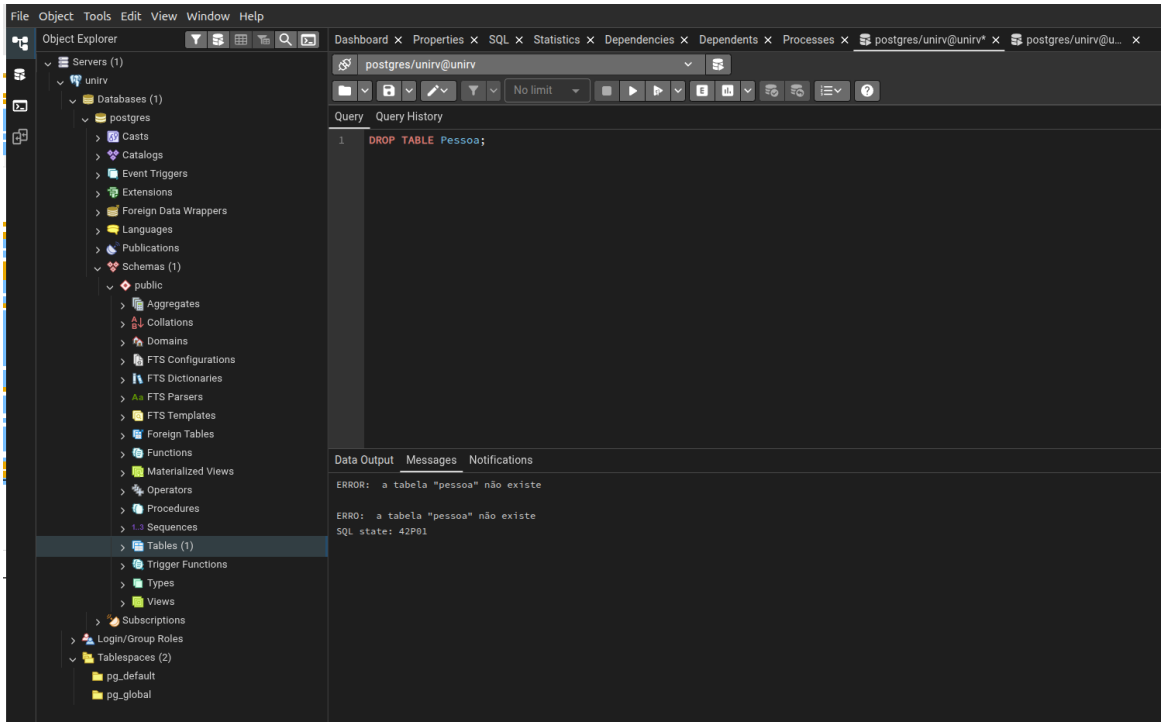


Figura 2.1: Interface do pgAdmin 4.

Enquanto o SQL é a linguagem utilizada para escrever comandos e realizar consultas diretamente no banco de dados, o pgAdmin 4 é o ambiente que possibilita criar e configurar servidores PostgreSQL, criar bancos de dados, tabelas e relacionamentos por meio de recursos gráficos, executar comandos SQL em um editor integrado, visualizar e editar dados em formato tabular, gerenciar usuários e permissões e monitorar as atividades do servidor.

O pgAdmin 4 pode ser executado tanto como aplicação desktop quanto por meio de navegador web. Sua interface é organizada em painéis, normalmente estruturados em um navegador de objetos (que apresenta a árvore de bancos de dados e seus componentes), um editor de consultas (Query Tool), no qual os comandos SQL são digitados e executados, e uma área destinada à exibição de resultados e mensagens do sistema.



Figura 2.2: Comparação entre pgAdmin 4 e PostgreSQL.

Há de se ressaltar que o pgAdmin 4 é uma ferramenta de administração e desenvolvimento para bancos de dados PostgreSQL, enquanto o PostgreSQL é o sistema gerenciador de banco de dados relacional em si. O pgAdmin 4 serve como uma interface gráfica para facilitar a interação com o PostgreSQL, permitindo que os usuários criem, configurem e gerenciem seus bancos de dados de maneira mais intuitiva, sem a necessidade de utilizar exclusivamente a linha de comando ou scripts SQL.

2.2 Iniciando o PostgreSQL no Windows

No ambiente Windows, o PostgreSQL é gerenciado como um serviço do sistema. Muitas vezes, após a instalação ou um reinício, o serviço pode estar interrompido, impedindo a conexão via pgAdmin ou terminal.

2.3 Método 1: Gerenciador de Serviços (Interface Gráfica)

Este é o método mais simples para usuários que preferem não utilizar linhas de comando.

1. Pressione as teclas **Win** + **R**, digite `services.msc` e aperte Enter.
2. Na lista de serviços, procure por **postgresql-x64-[versão]** (ex: postgresql-x64-16).
3. Clique com o botão direito sobre o nome do serviço.
4. Selecione **Iniciar** (Start) ou **Reiniciar** (Restart).

2.4 Método 2: Linha de Comando (Prompt/PowerShell)

Para ativar o serviço via terminal, você deve executar o Prompt de Comando como **Administrador**.

2.4.1 Iniciar o Serviço

SQL

```
net start postgresql-x64-16
```

2.4.2 Parar o Serviço

SQL

```
net stop postgresql-x64-16
```

**Nota: Substitua "16"pela versão que você instalou.*

2.5 Verificação de Variáveis de Ambiente

Para utilizar o comando `psql` em qualquer lugar do terminal, o caminho da pasta `bin` do PostgreSQL deve estar no **PATH** do sistema.

- Caminho padrão: `C:\Program Files\PostgreSQL\16\bin`

2.6 Teste de Conexão

Após ativar o serviço, teste a conexão no terminal:

SQL

```
psql -U postgres
```

O sistema solicitará a senha definida durante a instalação.

2.7 A Porta Padrão (5432)

O PostgreSQL utiliza, por padrão, a porta **5432** para comunicações TCP/IP. Esta porta é o "endereço do guichê"onde o serviço escuta as requisições de clientes como o pgAdmin, DBeaver ou aplicações Java/Python.

- **Localização do arquivo:** A porta pode ser alterada no arquivo `postgresql.conf`, na linha `port = 5432`.
- **Firewall:** Certifique-se de que a porta 5432 está liberada nas regras de entrada do Windows se for acessar o banco de outra máquina.

2.8 Credenciais de Acesso

Para este ambiente, utilizaremos as seguintes credenciais:

Parâmetro	Valor
Usuário	`unirv`
Senha	`unirv`
Porta	5432
Host	localhost

Tabela 2.1: Resumo das credenciais de conexão.

2.9 Comando para Criação do Usuário

Como o PostgreSQL inicia apenas com o usuário `postgres`, é necessário executar o comando abaixo para habilitar o novo usuário:

SQL

```sql
-- Criar usuario com senha e privilégios de criação de banco
CREATE USER unirv WITH PASSWORD 'unirv' CREATEDB;

-- Caso queira que ele seja um superusuario (uso acadêmico)
ALTER USER unirv WITH SUPERUSER;
```

2.10 String de Conexão (URL)

Em muitas linguagens de programação, a conexão é feita através de uma única linha:

`postgresql://unirv:unirv@localhost:5432/nome_do_banco`

2.11 Atividade Prática: Modelagem de Dados

Exercício Dirigido com o Professor

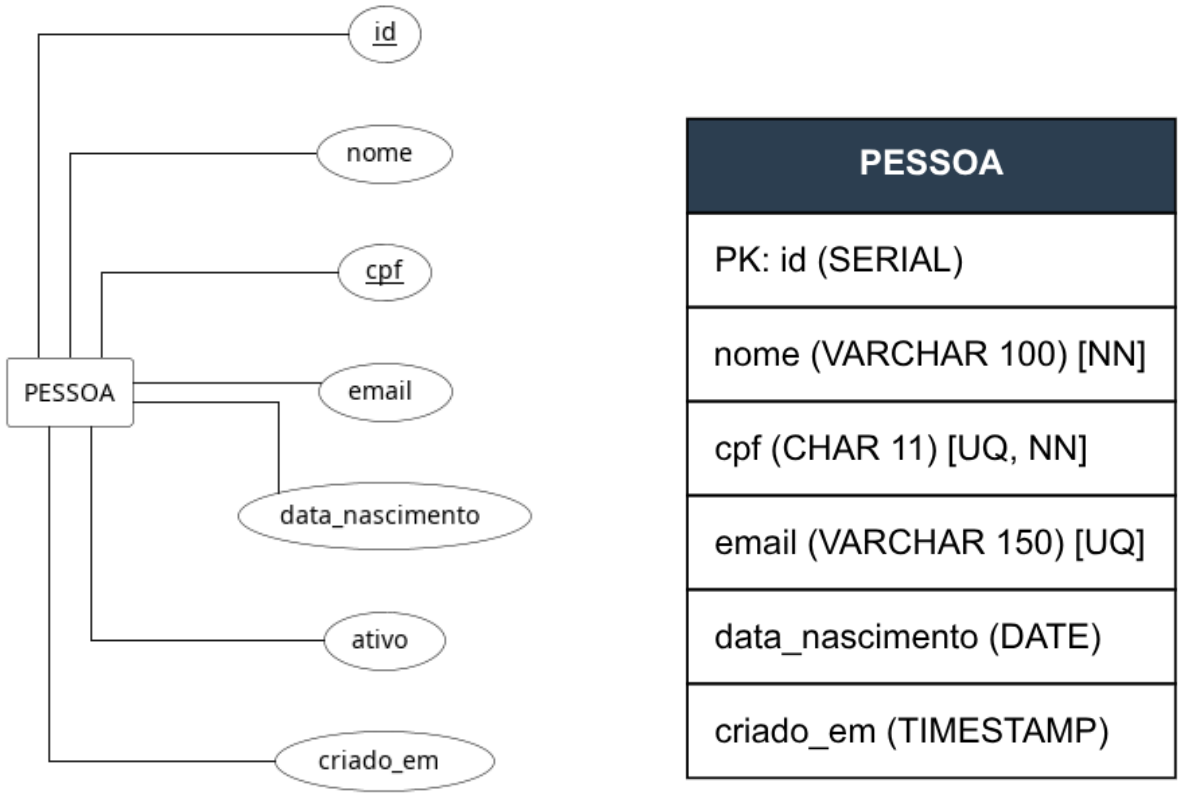
Nesta etapa, realizaremos a modelagem lógica da nossa primeira entidade. Sob a orientação do professor, analise os atributos necessários para representar uma **Pessoa** no sistema.
O objetivo é compreender como as regras de negócio do mundo real se transformam em restrições (*constraints*) no PostgreSQL.

2.11.1 Estrutura Lógica da Tabela `pessoa`

Abaixo, detalhamos a estrutura que será implementada. Preste atenção aos tipos de dados e às propriedades de cada campo:

- **id**: Chave Primária (*Primary Key*). Será do tipo `SERIAL` para garantir que o identificador seja autoincrementado e exclusivo.
- **nome**: Nome completo. Campo **obrigatório** (*NOT NULL*), definido como `VARCHAR(100)`.
- **cpf**: Cadastro de Pessoa Física. Deve ser **único** (*UNIQUE*) e **obrigatório**, garantindo que não haja duplicidade de registros.
- **email**: Endereço eletrônico. Também deve ser **único**, permitindo o uso como login ou canal de comunicação exclusivo.
- **data_nascimento**: Campo do tipo `DATE`, essencial para o cálculo de idade e validações de maioridade.
- **criado_em**: Registro automático de auditoria. Armazena a data e hora exata em que o registro foi inserido no banco.

Nota para o aluno: *Durante a explicação do professor, anote quais desses campos poderiam ser opcionais em um cenário real e quais devem ser sempre protegidos por restrições de integridade.*



(a) Diagrama ER (b) Diagrama Lógico

Figura 2.3: Modelagem da Entidade Pessoa: Visão ER e Lógica.

2.12 Script SQL de Criação

O código abaixo executa a criação da tabela no schema `public`.

SQL

```sql
CREATE TABLE pessoa (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    email VARCHAR(150) UNIQUE,
    data_nascimento DATE,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Adicionar um comentário na tabela para documentação

COMMENT ON TABLE pessoa IS 'Tabela que armazena dados cadastrais de
↪  pessoas físicas';
```

2.13 Operações Básicas (DML)

2.13.1 Inserção de Dados

Para inserir um novo registro, utilize o comando abaixo:

SQL

```sql
INSERT INTO pessoa (nome, cpf, email, data_nascimento)
VALUES ('João Silva', '12345678901', 'joao@email.com', '1990-05-15');
```

2.13.2 Consulta

Para listar todas as pessoas cadastradas:

SQL

```sql
SELECT id, nome, email FROM pessoa WHERE ativo = TRUE;
```

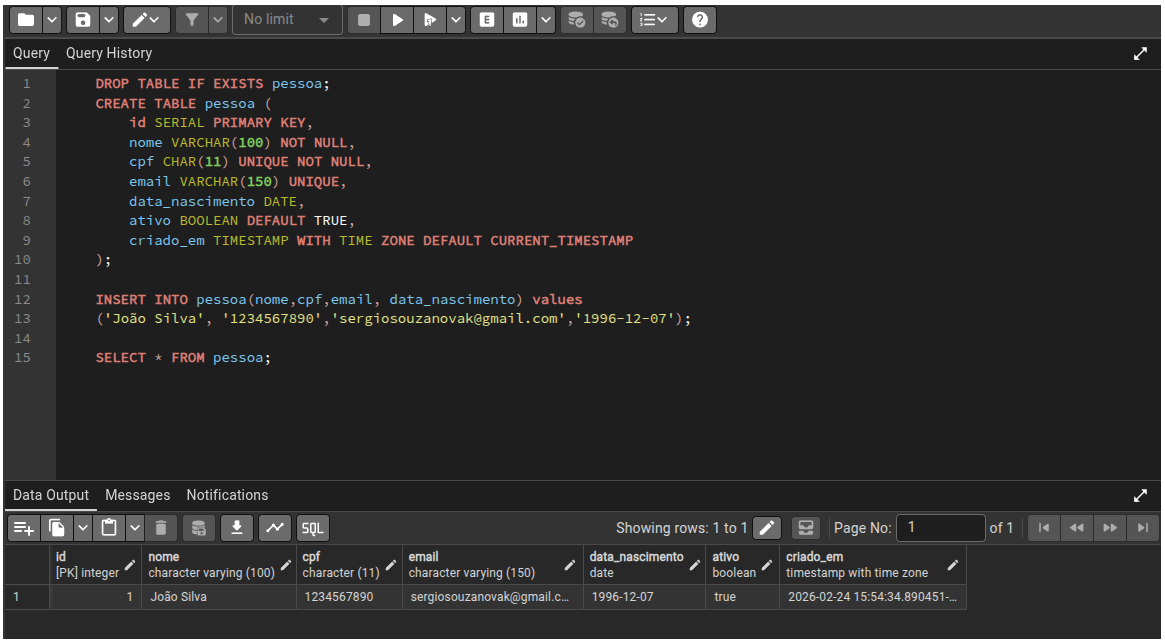


Figura 2.4: Execução do comando SELECT na tabela pessoa.

2.14 Comando SELECT: Estrutura e Funcionalidades

SQL

```sql
SELECT nome, salario
FROM funcionarios
WHERE salario > 30020
ORDER BY salario DESC;
```

O comando `SELECT` é utilizado para realizar consultas em um banco de dados relacional. Sua principal finalidade é recuperar dados armazenados em uma ou mais tabelas, permitindo ao usuário visualizar informações específicas conforme determinados critérios.

A estrutura básica do comando é:

SQL

```sql
SELECT coluna1, coluna2, ...
FROM nome_da_tabela;
```

Nesse formato, especificam-se as colunas que se deseja visualizar após a palavra-chave `SELECT`, enquanto a cláusula `FROM` indica a tabela de onde os dados serão obtidos. Caso seja necessário retornar todas as colunas da tabela, pode-se utilizar o caractere curinga `*`, como no exemplo:

SQL

```sql
SELECT * FROM alunos;
```

Além disso, o comando `SELECT` pode ser combinado com outras cláusulas para refinar

a consulta. Entre as mais utilizadas estão:

- `WHERE`: filtra registros de acordo com uma condição lógica;
- `ORDER BY`: organiza os resultados em ordem crescente ou decrescente;
- `GROUP BY`: agrupa registros para aplicação de funções agregadas;
- `HAVING`: define condições sobre grupos;
- `LIMIT`: restringe a quantidade de registros retornados (em alguns SGBDs).

Por exemplo:

SQL

```sql
SELECT nome, idade
FROM alunos
WHERE idade > 18
ORDER BY idade DESC;
```

Nesse caso, a consulta retorna apenas os alunos com idade superior a 18 anos, organizando o resultado em ordem decrescente de idade.Portanto, o comando `SELECT` é a base das operações de consulta em SQL, sendo essencial para a extração e análise de dados em sistemas de gerenciamento de bancos de dados relacionais.

2.14.1 Praticando Filtros e Ordenação: WHERE e ORDER BY

Exercício Dirigido com o Professor: Refinando a Busca

Em sistemas reais, um banco de dados pode conter milhões de registros. Retornar todos os dados de uma vez não é apenas ineficiente, mas muitas vezes inútil para o usuário final. Sob a orientação do professor, vamos testar na nossa tabela `Pessoa` como extrair apenas a informação exata que precisamos.

2.14.2 A Cláusula `WHERE` (Onde)

A cláusula `WHERE` atua como um "peneira". Ela avalia cada linha da tabela e retorna apenas aquelas que satisfazem uma condição lógica. Podemos usar operadores de comparação como `=`, `>`, `<`, `>=`, `<=` e `<>` (diferente).

Por exemplo, se quisermos encontrar apenas os usuários que estão com o cadastro ativo em nosso sistema:

SQL

```sql
-- O banco de dados retornará apenas as linhas onde 'ativo' for verdadeiro
SELECT nome, email
FROM pessoa
WHERE ativo = TRUE;
```

Outro exemplo comum é o uso com datas. Vamos buscar todas as pessoas que nasceram após o ano 2000:

SQL

```sql
SELECT nome, data_nascimento
FROM pessoa
WHERE data_nascimento > '2000-12-31';
```

2.14.3 A Cláusula `ORDER BY` (Ordenar Por)

Enquanto o `WHERE` decide *quais* linhas serão exibidas, o `ORDER BY` decide *em qual ordem* elas aparecerão na tela. A ordenação pode ser feita em duas direções:

- `ASC` (Ascendente): Ordem crescente (A-Z, 0-9, datas mais antigas para as mais recentes). Se você não especificar a direção, o SQL usará `ASC` por padrão.
- `DESC` (Descendente): Ordem decrescente (Z-A, 9-0, datas mais recentes para as mais antigas).

Se o professor pedir para organizar a lista de pessoas em ordem alfabética inversa (de Z a A), o comando será:

SQL

```sql
SELECT nome, cpf
FROM pessoa
ORDER BY nome DESC;
```

Juntando as Peças

O verdadeiro poder do SQL aparece quando combinamos essas cláusulas. É importante notar que **a ordem dos comandos importa**: o `WHERE` sempre deve vir antes do `ORDER BY`.

Vamos resolver um cenário prático: *"Liste o nome e a data de nascimento de todas as pessoas com cadastro ativo, ordenando das mais velhas para as mais novas"*.

SQL

```sql
SELECT nome, data_nascimento
FROM pessoa
WHERE ativo = TRUE
ORDER BY data_nascimento ASC;
```

2.14.4 Restringindo e Agrupando Resultados: LIMIT e HAVING

Exercício Dirigido com o Professor: Top Resultados e Agrupamentos

À medida que nosso banco de dados cresce, algumas perguntas mudam. Em vez de perguntar "Quais são todos os usuários?", passamos a perguntar "Quais são os 5 usuários mais recentes?"ou "Quantos usuários ativos temos em comparação aos inativos?". Acompanhe com o professor como responder a essas perguntas de negócios.

A Cláusula `LIMIT` (Limite de Linhas)

A cláusula `LIMIT` é extremamente útil para **paginação** (como quando você avança para a "página 2"de uma pesquisa no Google) ou para buscar os resultados do "Top N"(os maiores, menores, mais recentes, etc.). Ela simplesmente corta o resultado final para exibir apenas a quantidade de linhas especificada.

Exemplo Prático: Se o professor pedir para listar apenas as **3 pessoas cadastradas mais recentemente**, precisamos ordenar pela data de criação de forma decrescente e, em seguida, aplicar o limite:

SQL

```sql
-- Traz os 3 registros mais novos
SELECT nome, criado_em
FROM pessoa
ORDER BY criado_em DESC
LIMIT 3;
```

A Cláusula `HAVING` (Filtro de Grupos)

Para entender o `HAVING`, primeiro precisamos lembrar do `GROUP BY` (Agrupar Por), que une registros que possuem valores idênticos em uma ou mais colunas, permitindo o uso de funções matemáticas como `COUNT()` (contar), `SUM()` (somar) ou `AVG()` (média).

A grande confusão dos alunos iniciantes é: *Qual a diferença entre WHERE e HAVING?*

- O `WHERE` filtra **linhas individuais** ANTES de serem agrupadas.
- O `HAVING` filtra **os grupos** DEPOIS que eles foram formados.

Exemplo Prático: Vamos agrupar as pessoas pelo status (`ativo` = `TRUE` ou `FALSE`) e contar quantas pessoas existem em cada grupo. Porém, o professor solicitou que a consulta mostre **apenas os grupos que possuam mais de 10 pessoas**.

Como a restrição "mais de 10"é baseada no resultado de uma contagem (um grupo), **não podemos usar o WHERE**. Usamos o `HAVING`:

SQL

```sql
SELECT ativo, COUNT(id) AS total_pessoas
FROM pessoa
GROUP BY ativo
HAVING COUNT(id) > 10;
```

Atenção à Ordem de Execução!

O banco de dados SQL segue uma ordem estrita para ler o seu comando. Se você for usar todas as cláusulas que aprendemos até agora em uma única consulta, a ordem **obrigatória** no código deve ser:

1. `SELECT` (O que mostrar)
2. `FROM` (De qual tabela)
3. `WHERE` (Filtro de linhas)
4. `GROUP BY` (Como agrupar)
5. `HAVING` (Filtro de grupos)
6. `ORDER BY` (Ordem de exibição final)
7. `LIMIT` (Quantidade final de linhas)

2.15 Estudo Dirigido: Agrupamentos e a Cláusula `HAVING`

Objetivo da Atividade

Neste exercício, vamos simular um sistema de vendas para entender como o SQL pode realizar cálculos complexos e filtrar resultados baseados em agregados (somas), algo que o `WHERE` não consegue fazer sozinho.

2.15.1 Passo 1: Criando o Cenário de Teste

O primeiro passo é criar uma tabela temporária. Tabelas do tipo `TEMP` existem apenas durante a sua sessão atual no banco de dados.

SQL

```sql
CREATE TABLE pessoa (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    email VARCHAR(150) UNIQUE,
    data_nascimento DATE,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Adicionar um comentário na tabela para documentação

COMMENT ON TABLE pessoa IS 'Tabela que armazena dados cadastrais de
↪  pessoas físicas';
```

2.15.2 Passo 2: Populando a Tabela

Vamos inserir dados que nos permitam comparar categorias diferentes. Observe que a categoria 'Eletrônicos' terá um valor total maior que 'Papelaria'.

SQL

```sql
-- 2. Inserir dados para teste
INSERT INTO vendas (produto, categoria, valor) VALUES
('Teclado', 'Eletrônicos', 80.00),
('Mouse',   'Eletrônicos', 50.00),  -- Soma Eletrônicos: 130.00
('Caderno', 'Papelaria',   15.00),
('Caneta',  'Papelaria',    5.00);  -- Soma Papelaria: 20.00
```

2.15.3 Passo 3: A Consulta de Faturamento

Agora, execute a query abaixo. O objetivo é listar o faturamento total por categoria, mas **excluir** do relatório qualquer categoria que tenha vendido menos de 100.00 no total.

SQL

```sql
SELECT nome, data_nascimento
FROM pessoa
WHERE ativo = TRUE
ORDER BY data_nascimento ASC;
```

Questionário de Reflexão

Após executar os comandos acima, discuta com o professor:

1. Por que a categoria 'Papelaria' não apareceu no resultado final?
2. Se mudarmos o valor do `HAVING` para `10.00`, o que acontece?
3. Tente substituir `HAVING SUM(valor) > 100.00` por `WHERE SUM(valor) > 100.00`. Qual erro o PostgreSQL retorna? Por que isso acontece?

2.15.4 Diferença Crucial: WHERE vs HAVING

Para nunca mais esquecer:

- **WHERE**: Filtra os dados **antes** da soma (ex: excluir o produto 'Caneta' antes de somar).
- **HAVING**: Filtra os dados **depois** da soma (ex: olhar o total da categoria e decidir se mostra ou não).

2.16 Análise Avançada: Funções de Agregação e Comportamento do Agrupamento

Nesta etapa do estudo dirigido, vamos analisar como o banco de dados se comporta quando alteramos a lógica da nossa "pergunta". O código abaixo prepara uma base com diferentes estados de atividade para testarmos a sensibilidade das funções `COUNT` e `SUM`.

2.16.1 Script de Preparação

Observe que estamos lidando com restrições de integridade (*Unique*) e valores padrão (*Default*).

SQL

```sql
DROP TABLE IF EXISTS PESSOA;

CREATE TABLE PESSOA (
    ID SERIAL PRIMARY KEY,
    NOME VARCHAR(100) NOT NULL,
    CPF CHAR(11) UNIQUE NOT NULL,
    EMAIL VARCHAR(150) UNIQUE,
    DATA_NASCIMENTO DATE,
    ATIVO BOOLEAN DEFAULT TRUE,
    CRIADO_EM TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Inserindo dois registros inativos
INSERT INTO PESSOA (NOME, CPF, EMAIL, DATA_NASCIMENTO, ATIVO)
VALUES ('João Silva', '12345678900', 'joao@email.com', '1996-12-07',
↪  false),
       ('Camila Santana', '12345678901', 'camila@email.com', '1996-12-07',
       ↪  false);

-- Inserindo um registro ativo (padrão)
INSERT INTO PESSOA (NOME, CPF, EMAIL, DATA_NASCIMENTO)
VALUES ('João Silv1a', '12345678902', 'joao2@email.com', '1996-12-07');
```

2.16.2 Discussão: COUNT vs SUM

A consulta original utiliza a função `COUNT(ID)`. Mas o que aconteceria se tentássemos usar `SUM`?

SQL

```sql
INSERT INTO nome_da_tabela (coluna1, coluna2, ...)
VALUES (valor1, valor2, ...);
```

Conceito Fundamental

- **COUNT(ID)**: Conta quantas **linhas** existem no grupo. Funciona com qualquer tipo de dado (números, texto, datas, booleanos). No nosso exemplo, resultará em 2 para *false* e 1 para *true*.
- **SUM(coluna)**: Tenta realizar uma **adição aritmética** dos valores. Só funciona com tipos de dados numéricos (inteiros, decimais).

2.16.3 O que acontece se alterarmos para SUM?

Se tentarmos rodar `SELECT ATIVO, SUM(ID) FROM PESSOA GROUP BY ATIVO;`, o banco somará os valores numéricos das Chaves Primárias.

Função	Lógica	Resultado (para ATIVO = false)
`COUNT(ID)`	1 + 1 (duas ocorrências)	**2**
`SUM(ID)`	ID 1 + ID 2 (soma dos valores)	**3**

Tabela 2.2: Diferença de resultado entre contar e somar.

Reflexão para o Aluno

Pergunta: Por que não poderíamos usar `SUM(NOME)`?

2.17 Tipos de Dados Básicos no PostgreSQL

A escolha do tipo de dado correto é uma das etapas mais importantes da modelagem. No PostgreSQL, cada coluna deve ter um tipo definido para que o SGBD saiba como armazenar e processar aquela informação.

2.17.1 Principais Categorias

Abaixo, apresentamos os tipos mais utilizados em aplicações comerciais e acadêmicas:

Categoria	Tipo SQL	Descrição / Exemplo
Numéricos	`INTEGER`	Números inteiros (ex: idade, quantidade).
	`DECIMAL` ou `NUMERIC`	Números reais com precisão (ex: salário, preço).
	`SERIAL`	Inteiro autoincrementado (usado para IDs).
Texto	`VARCHAR(n)`	Texto de tamanho variável com limite n.
	`CHAR(n)`	Texto de tamanho fixo (ex: CPF, UF).
	`TEXT`	Texto de tamanho ilimitado (ex: descrições).
Data/Hora	`DATE`	Apenas data (AAAA-MM-DD).
	`TIMESTAMP`	Data e hora combinadas.
Lógicos	`BOOLEAN`	Valores verdadeiro (`TRUE`) ou falso (`FALSE`).

Tabela 2.3: Tipos de dados fundamentais no PostgreSQL.

2.17.2 Diferenças Importantes para o Desenvolvedor

Dica do Professor: CHAR vs VARCHAR

Uma dúvida comum é quando usar `CHAR` ou `VARCHAR`.

- Use `CHAR(n)` para dados que **sempre** têm o mesmo tamanho, como um campo *Estado* (UF) que sempre terá 2 letras ou um *CPF* com 11 dígitos.
- Use `VARCHAR(n)` para nomes, e-mails ou endereços, onde o tamanho muda de uma pessoa para outra. Isso economiza espaço de armazenamento.

2.17.3 Tipos Numéricos com Precisão

Para lidar com dinheiro (moeda), nunca utilize tipos de ponto flutuante (*float*), pois eles podem apresentar arredondamentos imprecisos. Utilize sempre:

SQL

```sql
SELECT nome FROM aluno WHERE cidade IN ('Rio Verde', 'Jataí', 'Mineiros');
SELECT aluno_id FROM matricula WHERE nota BETWEEN 6.0 AND 8.5;
```

2.18 Criação de Tabelas: O Comando `CREATE TABLE`

Após o planejamento no Diagrama Entidade-Relacionamento (ER), o próximo passo é a implementação física no SGBD. Para isso, utilizamos a linguagem de definição de dados (*Data Definition Language* - DDL) através do comando `CREATE TABLE`.

Este comando define não apenas o nome da tabela, mas também quais colunas ela terá, seus tipos de dados e suas regras de integridade.

2.18.1 Sintaxe Básica

A estrutura fundamental do comando segue o padrão abaixo:

SQL

```sql
-- Rastreando todos os alunos provados merecedores englobando e atestando
↪  que toda média final particular obtida unicamente foi auferida
↪  superando a escala limite baseada estritamente pontuada referenciada
↪  englobante e acima de sólidos 7 pontos corridos:
SELECT aluno_id, AVG(nota) AS media
FROM matricula
GROUP BY aluno_id
HAVING AVG(nota) > 7.0;
```

2.18.2 Restrições de Integridade (*Constraints*)

As restrições são regras aplicadas às colunas para garantir que os dados inseridos sejam válidos e consistentes. Sem elas, o banco de dados correria o risco de ter informações duplicadas ou incompletas.

As principais restrições são:

- `PRIMARY KEY`: Identifica unicamente cada registro. Não permite valores nulos nem repetidos.
- `NOT NULL`: Garante que a coluna não aceite valores vazios (campo obrigatório).
- `UNIQUE`: Garante que todos os valores na coluna sejam distintos (ex: CPF, e-mail).
- `DEFAULT`: Define um valor padrão caso nenhum seja informado na inserção.

2.18.3 Exemplo Comentado

Vamos analisar a criação da tabela `produto`, aplicando os conceitos aprendidos:

SQL

```sql
CREATE TABLE produto (
    id SERIAL PRIMARY KEY,            -- ID autoincrementado e chave
    ↪ primaria
    nome VARCHAR(100) NOT NULL,       -- Nome obrigatorio
    preco DECIMAL(10,2) NOT NULL,     -- Preco com 2 casas decimais
    estoque INTEGER DEFAULT 0,        -- Se nao informado, comeca com 0
    sku CHAR(8) UNIQUE                -- Codigo unico de produto
);
```

Dica do Professor: Planejamento

Antes de digitar o comando no terminal, sempre valide se os tipos de dados escolhidos suportam as necessidades do negócio. Mudar a estrutura de uma tabela (`ALTER TABLE`) depois que ela já possui milhões de registros é uma tarefa muito mais complexa e arriscada.

2.19 Inserção de Dados: O Comando `INSERT INTO`

Após criar a estrutura das tabelas com o `CREATE TABLE`, o próximo passo é popular o banco de dados. O comando `INSERT` é utilizado para inserir novos registros (linhas) em uma tabela existente.

2.19.1 Sintaxe Básica

Existem duas formas principais de utilizar este comando. A mais segura e recomendada é especificar explicitamente as colunas onde os dados serão inseridos:

SQL

```
INSERT INTO nome_da_tabela (coluna1, coluna2, ...)
VALUES (valor1, valor2, ...);
```

2.19.2 Regras Cruciais para Inserção

Para que a inserção ocorra sem erros, o desenvolvedor deve seguir algumas regras fundamentais de sintaxe e integridade:

- **Textos e Datas**: Devem sempre estar entre aspas simples (ex: `'João Silva'`, `'2024-05-10'`).
- **Números**: Não utilizam aspas (ex: `1500.50`, `25`).
- **Ordem**: A ordem dos valores no `VALUES` deve ser exatamente a mesma das colunas listadas.
- **Campos Obrigatórios**: Colunas com a restrição `NOT NULL` devem obrigatoriamente receber um valor, a menos que possuam um `DEFAULT`.

2.19.3 Prática: Inserindo Dados na Tabela Pessoa

Vamos aplicar o comando à tabela que modelamos anteriormente. Observe que não precisamos inserir o campo `ID`, pois ele é do tipo `SERIAL` e o banco o gera automaticamente.

SQL

```
-- Insercao especificando as colunas
INSERT INTO pessoa (nome, cpf, email, data_nascimento)
VALUES ('Ana Souza', '12345678910', 'ana@email.com', '1992-08-24');

-- Insercao de multiplos registros em um unico comando
INSERT INTO pessoa (nome, cpf, email)
VALUES
    ('Carlos Luz', '98765432100', 'carlos@email.com'),
    ('Beatriz Vaz', '55544433322', 'beatriz@email.com');
```

Atenção: Erros de Restrição

Se você tentar inserir um CPF que já existe ou deixar de preencher um campo `NOT NULL`, o PostgreSQL impedirá a operação e retornará um erro de **violação de restrição**. Isso garante que o banco de dados nunca aceite dados inválidos ou "sujos".

Dica do Professor: Omissions

Campos que possuem `DEFAULT` (como o campo `ativo` ou `criado_em` da nossa tabela) podem ser omitidos no `INSERT`. O PostgreSQL preencherá automaticamente com o valor padrão definido na criação da tabela.

Capítulo 3

O Comando SELECT: Consultando Dados

O comando `SELECT` é a ferramenta fundamental para extrair informações do banco de dados. Diferente das operações `INSERT`, `UPDATE` e `DELETE`, o `SELECT` apenas realiza a leitura dos dados. Toda a lógica de extração e cruzamento de informações numa aplicação moderna, desde relatórios gerenciais até tabelas de dashboards, depende substancialmente da construção correta desse comando.

Pense no banco de dados como uma biblioteca estruturada. O `SELECT` é a forma como você descreve **o que** quer encontrar e **como** deseja que esses dados sejam apresentados e organizados. O SGBD fará o trabalho pesado de varrer milhares de registros e entregar apenas o resultado desejado.

3.1 Sintaxe e Cenário de Exemplo

A estrutura base para toda consulta em SQL segue uma ordem obrigatória de declaração das cláusulas. Cada cláusula desempenha um papel específico no processamento da busca:

SQL

```sql
SELECT  [DISTINCT] colunas    -- O que mostrar (colunas)
FROM    tabela                -- De onde buscar (tabelas)
WHERE   condicao              -- Filtro de linhas (registros)
GROUP BY coluna               -- Agrupamento
HAVING  condicao_grupo        -- Filtro sobre grupos
ORDER BY coluna [ASC|DESC]    -- Ordem de exibição
LIMIT   quantidade;           -- Limite de linhas retornadas
```

Neste capítulo, utilizaremos três tabelas de um modelo acadêmico simples para ilustrar os conceitos:

- **aluno**: `id`, `nome`, `cidade`, `email`, `data_nascimento`, `ativo`.
- **curso**: `id`, `nome`, `area`.
- **matricula**: `id`, `aluno_id`, `curso_id`, `nota`, `ano`.

3.2 Buscando Dados: SELECT Básico

A forma mais simples de recuperar dados consiste em listar as colunas desejadas separadas por vírgula logo após o `SELECT`, e indicar a tabela alvo na cláusula `FROM`.

SQL

```sql
-- Recuperando colunas específicas:
SELECT nome, cidade FROM aluno;

-- Recuperando todas as informações (curinga *):
SELECT * FROM aluno;
```

Dica de Ouro

O uso do `SELECT *` em sistemas corporativos de produção não é recomendado, pois extrai colunas desnecessárias e sobrecarrega a rede. Especifique apenas as colunas que sua interface ou relatório realmente precisa.

3.2.1 Apelidos com AS e Valores Únicos com DISTINCT

Podemos renomear as colunas exibidas utilizando a palavra-chave `AS`. Esta alteração afeta apenas a exibição do resultado e não modifica a tabela original. Quando necessitamos apenas das ocorrências únicas de uma coluna (sem dados repetidos), aplicamos a cláusula `DISTINCT`.

SQL

```sql
-- Renomeando colunas para relatórios:
SELECT nome AS "Nome Completo", cidade AS "Naturalidade" FROM aluno;

-- Listando cidades únicas cadastradas, ignorando as duplicatas:
SELECT DISTINCT cidade FROM aluno;
```

3.3 Filtrando Linhas com WHERE

A cláusula `WHERE` atua como uma peneira, aplicando condições lógicas linha por linha. Apenas os registros cujo teste retorne verdadeiro (`TRUE`) farão parte do pacote final enviado ao usuário.

Os operadores básicos incluem igualdade (=), diferença (<> ou !=), maior que (>), menor que (<), e suas variantes (>=, <=).

SQL

```sql
-- Exibe apenas registros onde a cidade equivale a Rio Verde
SELECT nome FROM aluno WHERE cidade = 'Rio Verde';
```

3.3.1 Lógica Múltipla: AND, OR e NOT

O SQL permite combinações avançadas utilizando `AND` (onde obrigatoriamente todas as premissas devem ser reais), `OR` (sendo suficiente apenas um lado provar-se verídico) e `NOT` (fator modificador que inverte a premissa).

SQL

```sql
-- Alunos ativos (E) moradores de Goiânia
SELECT nome FROM aluno WHERE cidade = 'Goiânia' AND ativo = TRUE;

-- Alunos não ativos de qualquer cidade
SELECT nome FROM aluno WHERE NOT ativo = TRUE;
```

3.3.2 Operadores Estruturais: IN e BETWEEN

Para checar contra uma pluralidade solta de múltiplos valores usa-se o termo `IN`. Já para abranger intervalos numéricos ou temporais perfeitamente, emprega-se `BETWEEN`, que garante a inclusão do teto e do piso no processo.

SQL

```sql
SELECT nome FROM aluno WHERE cidade IN ('Rio Verde', 'Jataí', 'Mineiros');
SELECT aluno_id FROM matricula WHERE nota BETWEEN 6.0 AND 8.5;
```

3.3.3 Busca Textual Avançada com LIKE

O operador textual de similaridade `LIKE` trabalha operando metacaracteres específicos. É de uso rotineiro nas barras de pesquisa e filtros gerenciais e telas de listagem.

- % Substitui um aglomerado de múltiplos caracteres em sequência (ou mesmo nenhum).
- _ Restringe a substituição unicamente para um exato caractere isolado.

SQL

```sql
-- Pessoas da família 'Lima' (não importa o que vem antes ou depois)
SELECT nome FROM aluno WHERE nome LIKE '%Lima%';

-- Nomes iniciados com a letra B
SELECT nome FROM aluno WHERE nome LIKE 'B%';
```

3.3.4 Lidando com a Ausência Absoluta: IS NULL

No ecossistema SQL, o elemento `NULL` aponta para uma ausência imutável de dado preenchido; ou seja, o campo não possui lastro, nunca foi editado ou se perdeu. Nunca verificamos `NULL` com operador matemático de igualdade (`=`), devendo invocar o `IS NULL` ou o reverso `IS NOT NULL`.

SQL

```sql
-- Detecção de cadastro com pendência de e-mail (em branco/nulo absoluto)
SELECT nome FROM aluno WHERE email IS NULL;
```

3.4 Ordenando e Limitando Resultados

Sem intervenção sistemática explícita, um retorno do SGBD pode apresentar linhas de maneira randômica, baseando-se no critério interno físico ditado por onde a linha se alojou na superfície de leitura do disco rígido.

3.4.1 Ordenação Descritiva com ORDER BY

Para consolidar análises claras, forçamos um direcionamento organizacional estrito aplicando `ORDER BY`. O compilador oferece o enquadramento linear natural crescente e alfanumérico padrão original referenciado pelas siglas (`ASC`) em contraposição ao modo invertido regressivo (`DESC`).

Podemos definir chaves primárias e secundárias preestabelecendo critérios rigorosos focados no desempate posicional em ocorrências conjuntas:

SQL

```sql
-- Agrupa blocos em ordem crescente pelas cidades adjacentes,
-- desempatando alunos residentes da mesma urbe pelo nome em ordem
↪  decrescente
SELECT nome, cidade FROM aluno ORDER BY cidade ASC, nome DESC;
```

3.4.2 Supressão e Segmentação Paginada: LIMIT e OFFSET

Como frequentemente as aplicações modernas exibem seus relatórios particionados de forma palatável amigavelmente em grades compostas por páginas numéricas de tabelas de interface. Controlamos precisamente esse tráfego de exibição mitigando excessos operacionais ao segmentar retornos via `LIMIT`. O termo atrelado `OFFSET` engloba o papel de pular, no bastidor interno da varredura operante executiva, os patamares listados de instâncias inicialmente contabilizadas que não competem à exibição da fração ativa solicitada.

SQL

```sql
-- Recuperando estritamente os cinco estudantes de maior potencial
↪  acadêmico
SELECT aluno_id, nota FROM matricula ORDER BY nota DESC LIMIT 5;

-- Mecânica focal paginada: Retorna cinco instâncias suprimindo os dez
↪  prévios
SELECT nome FROM aluno ORDER BY nome LIMIT 5 OFFSET 10;
```

3.5 Agrupando Dados e Agregações

Em situações analíticas focadas onde a averiguação corporativa exige apurações sintéticas com sumarização extensiva macro, deixamos de lidar pontualmente com cada registro atomizado singular. Aplicamos ativamente equações providas pelas rotuladas "funções de agregação": A modalidade de varredura cumulativa enumerando tuplas atende por `COUNT()`, as prospecções orientadas focadas explicitamente a identificar extremos tetos ou abismos pisos invocamos `MAX()` atrelado concomitantemente na outra polaridade avaliativa com e `MIN()`, e a consolidação focada em sintetizar a raiz algébrica acumulativa e média aritmética providas pelos termos `SUM()` e `AVG()`.

SQL

```sql
-- Encontra a quantia monetária máxima, mínima e condensa todo o resto na
↪  média
SELECT MAX(nota), MIN(nota), AVG(nota) FROM matricula;
```

3.5.1 Segmentando Agrupamentos Estritos: GROUP BY

O SGBD atinge seu pleno potencial ao acoplar funções agregativas no enquadramento fragmentador departamental lógico `GROUP BY`. Ele obrigatoriamente segmenta todos os registros matriz que se assemelham no crivo da restrição alocando e separando tais conglomerados de tal formato que as agregações computáveis processam de maneira fragmentada calculando de subgrupo a subgrupo, resultando extratos individuais avaliativos:

SQL

```sql
SELECT nome, idade
FROM alunos
WHERE idade > 18;
```

3.5.2 A Peneira Superior de Aglomerados Analíticos: HAVING

Lembre-se imperativamente das camadas constituidoras de modelagem matricial relacional, não utilizamos a condicional delimitadora `WHERE` no foco analítico se estivermos ambicionando promover censura com avaliações diretas e calcificadas em base do teto ou sínteses aglutinadas recém computadas. O preceito clássico matriz atua previamente separando lixo do material orgânico, ao passo e tempo que todo exame analítico executivo consolidado provindo de extrato originado das próprias funções agregadoras deverá de fato estar posicionado englobado restritamente abrigado unicamente sob as sentenças reguladoras provadas no cerne da estrutura avançada condicionadora expressa sob o código do termo e chancela nominativa `HAVING`.

SQL

```sql
-- Rastreando todos os alunos provados merecedores englobando e atestando
↪  que toda média final particular obtida unicamente foi auferida
↪  superando a escala limite baseada estritamente pontuada referenciada
↪  englobante e acima de sólidos 7 pontos corridos:
SELECT aluno_id, AVG(nota) AS media
FROM matricula
GROUP BY aluno_id
HAVING AVG(nota) > 7.0;
```

3.6 Junções Matemáticas Complexas: JOINs

A verdadeira inteligência processual reside num princípio lógico fundacional arquitetônico da sistemática contida nativamente do princípio elementar das bases de dados relacionais ativas de fato, interligar entidades fracionadas independentes englobando seus vetores apontadores para reconstruir uma visão sintética compreensiva global integrada e completa. O operador analítico mesclador interno matriz elementar executor é chamado formal e universalmente conhecido pela denominação estrita unificada condutiva contígua rotulada unicamente e referenciada amplamente da clássica chave operacional `JOIN`.

3.6.1 O Acoplamento Perfeito Simétrico: INNER JOIN

Tratando do caso restritivo simétrico principal balizador denominado na essência englobante `INNER JOIN`, operamos estritos elos de correlação espelhada em consonância atrelada unicamente nas linhas dotadas explicitamente e preenchidas restritamente que provam paridade absoluta equivalência mútua presente simultaneamente interconectada referencial restritivamente idêntica em ambos espectros lados tabelados simultaneamente inseridos postos englobados:

SQL

```sql
net stop postgresql-x64-16
```

3.6.2 O Espelhamento Total Elementar Origético: LEFT JOIN

Abordagens direcionadas assimétricas englobam preceito de totalidade, na declaração referida base contida regrada imperativamente declarada esquerda contígua analítica unicamente `LEFT JOIN` engloba na premissa elementar fundamental manter sem perda excludente restritiva pontuais todas representatividades listadas originalmente advindas do polo e quadrante matriz declarado contido do lado alocado primeiro esquerdista (tabela referenciada unicamente na estrutura preestabelecida de declarações em matriz de consulta englobante do núcleo premissa `FROM`). Se inexistirem correspondências ligantes diretas simétricas na estrutura tabela do lado justaposto oposto vizinho direita atrelada, este completará unicamente entregando valor vazio limitante expresso condicional contido formatado pontual vazio exposto nulo `NULL` garantindo entrega inteira.

SQL

```sql
-- Lista sem omissões por excepcionalidade todos formandos expostos
↪  evidenciando até casos faltantes pendentes isolados:
SELECT a.nome, m.nota
FROM aluno AS a
LEFT JOIN matricula AS m ON a.id = m.aluno_id;
```

3.7 Ordem Linear Processada Compiladora

Num fluxo processual restritivo base focado sequencial primário de execuções lógicas avaliativas condicionadas imposto, temos a cronologia base executável:

1. **FROM/JOIN**: Mescla as bases.
2. **WHERE**: Filtragem inicial linha a linha.
3. **GROUP BY**: Junção e condensação em blocos.
4. **HAVING**: Filtragem profunda nos blocos.
5. **SELECT**: Cálculos explícitos.
6. **DISTINCT**: Expurgo de repetições textuais geradas.
7. **ORDER BY**: Estética e alinhamento visual final resultante.
8. **LIMIT / OFFSET**: Paginação de recortes avaliativos seccionais.

3.8 Laboratório e Práticas Orientadas

Exercícios - Sintaxe DQL de Estruturas Restritivas Contidas

1. Desenvolva relatórios apontadores apenas englobando instâncias nativas da cidade 'Rio Verde'.
2. Formule pesquisa estrita listando numerado os nichos por municípios ativando o comando aglomerante consolidado prático referenciado contido focado na matriz em (GROUP BY).
3. Restrinja visual final das apurações do retorno limitante estritamente prático focados englobando restritivamente em apurações de exibir formatados apenas cinco exemplares paginados iniciais.
4. Interligue simetricamente o consolidado dos estudantes alinhados as grades contidas mescladas apontadas focando relacionando ativamente no limitante (INNER JOIN).
5. Estabeleça restrições posteriores limítrofes baseando apurações nas extremidades estritas (HAVING) englobando restritivamente o polo de avaliação da média > 5.0.